Karadeniz Teknik Üniversitesi
Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Hızlı Başvuru Kağıdı #3: Modern C++ ve STL Uygulamaları

Yazılım Geliştirme II – Hafta 3

Ranges ve Views

Ranges Temelleri

```
#include <ranges>   #include <algorithm>
```

Ranges algoritmaları doğrudan kapsayıcı alır (begin/end gerekmez).

Concept	Anlam
input_range	Okunabilir
forward_range	Çok geçişli
bidirectional_range	Geri hareket
random_access_range	Aritmetik

Projeksiyonlar: `ranges::sort(v, {}, &Ogr::isim)`

Temel Ranges Algoritmaları

```
std::ranges::sort(v);                    // kucuk->buyuk
std::ranges::sort(v, std::greater{});// buyuk->kucuk
auto it = std::ranges::find(v, 42);
auto it = std::ranges::find_if(v,
    [](int x){ return x > 50; });
auto n = std::ranges::count_if(v, pred);
```

Views Pipeline

```
#include <ranges>
namespace rv = std::views;
auto r = v | rv::filter(pred)
           | rv::transform(fn)
           | rv::take(5);
for (int x : r) std::cout << x;
```

Views Kataloğu

View	Açıklama
filter	Koşulu sağlayanlar
transform	Dönüştürme uygula
take(n)	İlk n eleman
drop(n)	İlk n'yi atla
reverse	Ters sıra
keys	map anahtarları
values	map değerleri

Views Tembel (Lazy)!

Veri kopyalamaz, anında işlemez. İterasyon sırasında talep üzerine hesaplanır. Birden fazla view zincirlense bile tek geçişte çalışır.

String İşlemleri ve Formatlama

std::string_view

```
#include <string_view>
std::string_view sv = "merhaba";
sv.substr(0, 3);          // "mer"
sv.starts_with("mer");    // true
sv.ends_with("ba");       // true
sv.find("hab");           // 3
// Dikkat: kaynak string yasamali!
```

string_view Tehlikesi!

Kaynak yok olunca dangling reference oluşur. Geçici string'den `string_view` almayın!
`auto sv = std::string_view(fonk());` → **Tehlike!**

std::format

```
#include <format>
std::format("Sayi: {}", 42);
std::format("{:>10}", "test");     // saga hizala
std::format("{:<10}", "test");     // sola hizala
std::format("{:^10}", "test");     // ortala
std::format("{:.2f}", 3.14159);    // 3.14
std::format("{:05d}", 42);         // 00042
std::format("{0} {1} {0}", "a","b"); // a b a
```

Format Belirteçleri

Belirteç	Anlam
{:d}	Tam sayı
{:f}	Ondalıklı sayı
{:s}	String
{:>N}	Sağa hizala (N genişlik)
{:<N}	Sola hizala
{:^N}	Ortala
{:0N}	Sıfır doldurma
{:.Nf}	N ondalık basamak

std::optional

```
#include <optional>
std::optional<int> a = 42;
std::optional<int> b; // veya nullopt
if (a) cout << *a;          // 42
a.value();                  // 42 (ya da istisna)
b.value_or(-1);             // -1
a.has_value();              // true
a.reset();                  // bos yap
```

std::variant

```
#include <variant>
std::variant<int, double, string> v = 42;
v.index();                      // 0
std::get<int>(v);               // 42
std::holds_alternative<int>(v); // true

// visitor pattern
std::visit([](auto&& val) {
    std::cout << val;
}, v);
```

İstisna Yönetimi

İstisna Hiyerarşisi

```
try {
    throw std::runtime_error("Hata!");
} catch (const std::runtime_error& e) {
    std::cerr << e.what();
} catch (const std::exception& e) {
    std::cerr << "Genel: " << e.what();
} catch (...) {
    std::cerr << "Bilinmeyen hata";
}
```

std::exception Ailesi

Tür	Alt Sınıflar
logic_error	invalid_argument
	out_of_range
	domain_error
runtime_error	overflow_error
	underflow_error
	range_error

bad_alloc, bad_cast, bad_optional_access

Özel İstisna Sınıfı

```
class OBYSHata : public std::runtime_error {
public:
    OBYSHata(const std::string& msg)
        : std::runtime_error(msg) {}
};
class ÖğrenciBulunamadi : public OBYSHata {
    int numara_;
public:
    ÖğrenciBulunamadi(int no)
        : OBYSHata("Öğrenci bulunamadi: "
          + std::to_string(no)), numara_(no) {}
    int numara() const { return numara_; }
};
```

noexcept

```
void guvenli() noexcept { /* ... */ }
// Istisna firlatirsa -> terminate()
// Derleyici optimizasyon yapabilir
bool b = noexcept(expr); // compile-time
```

overloaded Deseni

```
template<class... Ts>
struct overloaded : Ts... {
    using Ts::operator()...;
};
// Kullanim:
std::visit(overloaded{
    [](int i)    { cout << "int: " << i; },
    [](double d) { cout << "dbl: " << d; },
    [](string s) { cout << "str: " << s; }
}, v);
```

Hata Yönetimi Seçim Rehberi

Değer yok → `optional`
Çok türlü sonuç → `variant`
Kurtarılabilir hata → `exception`
Kritik hata → `terminate` / `abort`
Performans kritik → hata kodu + `noexcept`